Knowledge-Based Systems 343 (2026) 116005

Contents lists available at ScienceDirect

Knowledge-Based Systems

journal homepage: www.elsevier.com/locate/knosys

Causal-Nest: A framework for automated causal discovery and inference

Gustavo F.V. de Oliveira *, Fabrício A. Silva, Marcus H.S. Mendes

IEF, Universidade Federal de Viçosa - Campus Florestal, Florestal, MG, Brazil

ARTICLE INFO

Keywords:
Causality
Causal discovery
Causality for ML
Causal feature engineering
Causal inference framework

ABSTRACT

Causal discovery offers a path for Machine Learning models to move beyond mere correlation to understand cause-and-effect relationships. However, its operational complexity remains a significant barrier for non-expert developers. This paper introduces *Causal-Nest*, a new, comprehensive framework that simplifies and operationalizes the causal analysis pipeline for non-experts. The contribution of the framework is demonstrated through an extensive, two-part evaluation.

First, we validate its theoretical correctness and diagnostic power using the well-known Sachs et al. dataset and an extended benchmark of seven real-world datasets with known causal constraints. Our results show that the proposed framework effectively assesses causal graph integrity, using metrics, including the novel Knowledge Integrity Score (KIS), to pinpoint structural discovery failures and statistical unreliability quantitatively.

Second, we demonstrate its practical utility in a common Machine Learning scenario where no such ground truth is available: telecommunications customer churn prediction. We show that using the proposed framework to guide a causality-driven feature engineering process yields a causal oversampled feature set that significantly improves the minority-class F1-Score of ensemble prediction models (e.g., CBC, GBMC).

This work validates *Causal-Nest* as a single, dual-purpose tool that successfully bridges the gap between rigorous causal validation and practical, performance-driven machine learning applications.

1. Introduction

Causation is the concept referring to the cause-and-effect relationship wherein one event (the cause) leads to another event (the effect) and establishes a direct connection or influence between them [1]. Artificial Intelligence (AI) models have demonstrated remarkable capabilities in establishing intricate connections and making accurate predictions from data. However, these models face challenges when confronted with unfamiliar data, leading to inaccurate causal relationships [2]. This limitation is particularly troubling in ethical AI, where biased associations are highly undesirable, such as the risk of incorrect associations between factors like race and criminal behavior, resulting in unjust outcomes [3]. Causal reasoning enables AI systems to progress beyond mere correlations and infer causal relationships, empowering them to make more informed decisions and predictions [4]. By understanding causality, AI models can distinguish between false connections and genuine causal relationships, leading to more accurate, ethical, and reliable outcomes through causality-based feature selection [5]. Therefore, AI models must consider cause-and-effect relationships, which is a key aspect for their adoption in sensitive fields such as self-driving vehicles [6], healthcare [7], justice [3], and finance [8].

The complexity of the field, with its requirements of domain knowledge, causal Directed Acyclic Graph (DAG) notation, Causal Discovery algorithms, Causal Inference estimators, and refuters, can be discouraging. While adopting causal analysis can enhance AI models, incorporating these concepts can be overwhelming and challenging due to the steep learning curve and the need to stay current with evolving content. The early stages of causal tools development pose a challenge to their widespread adoption. The field is relatively new, with the most popular and corporate-backed packages released within the last five years. Additionally, essential performance features such as time limits and parallelism have not yet been universally implemented, which poses a significant challenge for practitioners building continuously learning AI systems.

To address these challenges, we propose *Causal-Nest* : a comprehensive framework that simplifies causal discovery for non-experts. We introduce the *Causal-Nest* architecture and validate its correctness and structural integrity. We perform an extensive benchmark analysis across seven diverse, real-world datasets where causal constraints are known. This allows us to use metrics such as the Knowledge Integrity Score (KIS) and the Priority Score to quantitatively assess the ability

* Corresponding author.
E-mail address: gustavo.viegas@ufv.br (G.F.V. de Oliveira).

https://doi.org/10.1016/j.knosys.2026.116005
Received 30 October 2025; Received in revised form 31 March 2026; Accepted 12 April 2026
Available online 17 April 2026
0950-7051/© 2026 The Author(s). Published by Elsevier B.V. This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/).

of the framework to produce structurally sound, reliable causal graphs. The main contributions of this work are as follows:

- **An end-to-end causal analysis framework:** We present *Causal-Nest*, an open-source Python framework that integrates twelve CD algorithms with causal effect estimation and refutation methods. Unlike existing tools that specialize in either discovery or inference, *Causal-Nest* provides a unified pipeline from raw data to validated causal conclusions, with built-in parallel execution and configurable time limits.
- **Novel metrics for causal graph assessment:** We introduce the Knowledge Integrity Score, a metric that quantifies adherence to domain knowledge constraints by combining forbidden edge violations and required edge compliance. We also propose the Priority Score algorithm, which ranks discovered causal graphs based on structural properties indicative of causal relevance to the target variable.
- **Automated algorithm selection:** We define rules to automatically filter applicable CD algorithms based on data characteristics (variable types, distribution, linearity), removing a significant knowledge barrier for non-expert users.
- **Comprehensive empirical validation:** We validate the framework on eight real-world datasets spanning diverse domains, demonstrating both its correctness against known ground truth and its diagnostic power in identifying structural failures and statistical unreliability.
- **Practical demonstration of causality-driven feature engineering:** Through a case study on telecommunications churn prediction, we show that using *Causal-Nest* to guide feature engineering yields causally-informed feature sets that improve minority-class F1-Scores for ensemble prediction models.

The remainder of this paper is structured as follows: Section 2 reviews related work. Section 3 details the framework architecture. Section 4 presents the benchmark validation results. Finally, Section 6 concludes the work.

2. Related work

This section discusses key components of a causality pipeline that systematically identify causal relationships from observational data. The main steps of the pipeline are causal discovery to uncover the causal graph and causal inference to estimate effects. To our knowledge, existing literature lacks comprehensive frameworks that can independently address all aspects of a causality problem without the need for manual algorithm selection, model prioritization, and parallel execution, which would typically necessitate a substantial understanding of causality from the user.

2.1. Causal Discovery (CD)

The process of extracting causality knowledge from data typically involves several key steps. It begins with problem formulation and the construction of a causal model using a DAG, where edges between nodes represent hypothesized causal pathways among identified variables. However, creating such a graph can be challenging without deep knowledge of the problem domain and an understanding of DAGnotation. Causal Discovery (CD) refers to the process of extracting these models from observational data. It serves as an alternative to manually building such a graph, offering several models that can be adopted under different circumstances. Problem solvers must thoroughly understand Causal Discovery (CD) algorithms to ensure a suitable model is applied [1].

CD methods often yield DAGs or undirected Bayesian networks, each presenting its computational complexities. Various methodologies exist for uncovering causal graph structures, each with distinct approaches and computational complexity. Bayesian Networks utilize probabilistic inference and learning techniques, employing algorithms such as Grow-Shrink (GS) [9] and Incremental Association Markov Blanket (IAMB) [10]. Conversely, constraint-based methods like the Peter-Clark (PC) algorithm [11] use statistical constraints to identify conditional independence relationships among variables. Score-based methods assess and compare different models using scoring criteria to select the optimal causal model [12]. More recently, continuous optimization-based methods have reformulated structure learning as a smooth, differentiable optimization problem subject to an acyclicity constraint, enabling gradient-based solvers to replace computationally intractable combinatorial search. Hybrid approaches that embed conditional independence tests as structural constraints within the continuous optimization objective have demonstrated improved robustness under small sample sizes and heterogeneous noise [13,14]. Continuous optimization has also been extended to causal representation learning, where it is embedded within autoencoder architectures to uncover invariant causal features in high-dimensional, nonlinear data for cross-domain classification tasks [15]. Several factors must be considered when choosing a CD algorithm, including computational complexity, data assumptions, and the ability to extract meaningful causal relationships. The selection criteria involve analyzing the data types, distribution assumptions, and linearity assumptions. Different methods are tailored for continuous or categorical data, handle Gaussian or non-Gaussian distributions differently, and require linear relationships between variables in some cases [12].

2.2. Causal Inference (CI)

Causal Inference (CI), which involves estimation and refutation, is a complex task that requires estimating causal effects based on a causal graph. Statistical estimators quantify causal effects between variables in a causal model. A wide range of estimators is available in the literature, including traditional methods such as linear regression [16] and more sophisticated machine learning techniques [17].

Each estimator applies different interpretations to the estimated causal effects, necessitating domain-specific expertise to interpret numerical values accurately and meaningfully. Validating the accuracy of the causal effect estimations depends on a comprehensive understanding of the data domain. Background knowledge is crucial for conducting refutation tests to scrutinize and potentially reject causal models that do not exhibit expected behaviors [1].

These tests involve various sensitivity analyses, including introducing random placebo variables instead of treatment features, adding random standard cause variables, or removing subsets from the data. These processes ensure the inferred causal relationships are robust and align with theoretical expectations [18].

2.3. Discussion

The principles of causal discovery and inference have been implemented in numerous open-source libraries. However, the existing ecosystem is highly fragmented, with tools typically specializing in one discrete component of the causal pipeline (e.g., discovery or estimation) rather than providing a seamless, end-to-end framework.

For the initial step of CD, several powerful libraries are available. **causal-learn** [19] is a prominent Python package implementing a wide array of CD algorithms, including constraint-based, score-based, and methods for non-linear or non-Gaussian data. Similarly, **pgmpy** [20] and the R package **bnlearn** [21] provide robust implementations for learning the structure of Bayesian networks. The **Tetrad** [22] platform remains a comprehensive, long-standing workbench for causal research. While algorithmically comprehensive, these tools are designed for causal researchers and present a significant barrier to adoption for non-experts. They require users to possess prerequisite knowledge to select an appropriate algorithm and correctly interpret the output,

Table 1
Comparison of causal analysis software tools. ✓ indicates the feature is supported.

Tool	Discovery	Estimation	Refutation	Parallel	Time-series
CDT [28]	✓	–	–	–	–
causal-learn [19]	✓	–	–	–	–
bnlearn [21]	✓	–	–	–	–
Tetrad [22]	✓	–	–	–	–
gCastle [29]	✓	–	–	✓	–
DoWhy [25]	–	✓	✓	–	–
EconML [23]	–	✓	–	–	–
CausalML [24]	–	✓	–	✓	–
CausalNex [30]	✓	✓	–	✓	–
Salesforce CausalAI [31]	✓	✓	–	✓	✓
Tigramite [26]	✓	✓	–	✓	✓
CausalFlow [27]	✓	–	–	–	✓
Causal-Nest	✓	✓	✓	✓	–

which is often a Markov equivalence class rather than a single, unambiguous DAG. Furthermore, they are discovery-focused and typically lack integrated modules for subsequent causal inference and estimation.

Conversely, a separate class of tools focuses on CI, assuming a causal graph is already known. Industry-developed libraries, such as **EconML** [23] from Microsoft and **CausalML** [24] from Uber, offer a rich toolkit of modern machine learning estimators, particularly for estimating heterogeneous treatment effects. These libraries are primarily *estimators* toolkits. They assume the user has already defined a causal graph, identified the causal estimand, and now seeks a method for statistical estimation. They do not provide guidance for a user starting from raw data without a predefined graph, nor do they typically include discovery modules.

A limited number of frameworks have attempted to unify the entire pipeline. The most notable is **DoWhy** [25] from Microsoft, which structures causal analysis into four conceptual steps: *Model, Identify, Estimate,* and *Refute*. This design is a significant contribution, as **DoWhy** is one of the few libraries to programmatically integrate refutation tests (such as placebo treatments or random common causes) as a core component of its API, directly addressing the validation stage of the pipeline. Despite its "end-to-end" design, the primary limitation of **DoWhy** is its *Model* step. The framework requires the user to provide the causal graph; thus, it does not perform causal discovery. While **DoWhy** can be manually integrated with a discovery library such as **causal-learn**, this reintroduces the knowledge barrier. A non-expert user must learn two separate, complex libraries and understand the non-trivial process of correctly passing the output of one (a potentially complex causal graph or equivalence class) as the input to the other.

Table 1 summarizes the capabilities of existing causal analysis tools across five key dimensions: causal discovery, causal effect estimation, refutation testing, parallel execution support, and time-series data support. As illustrated, no existing framework provides comprehensive support across all stages of the causal analysis pipeline with integrated refutation. Discovery-focused tools (CDT, causal-learn, bnlearn, Tetrad, gCastle) lack estimation and refutation capabilities, while inference-focused tools (DoWhy, EconML, CausalML) assume the causal graph is already provided. Time-series specialized libraries such as Tigramite [26] provide both discovery and causal effect estimation modules, but do not integrate refutation testing. CausalFlow [27] focuses on discovery without estimation or refutation. This fragmentation compels practitioners to manually integrate multiple libraries, increasing the barrier to entry and the risk of methodological errors.

In summary, the current open-source landscape presents a dichotomy: users must either select a specialized discovery tool that requires in-depth statistical knowledge or an inference tool that demands prior domain knowledge to construct the causal model. A clear gap exists for an integrated framework that abstracts this complexity and guides a user with minimal causality knowledge, such as a data engineer, through the complete pipeline from data ingestion to discovery, estimation, and refutation.

3. *Causal-Nest*

This work proposes a novel framework named *Causal-Nest*.[1] This framework includes a user-friendly interface for easier adoption and automatically manages the many rules and restrictions present in the field of causality. Fig. 1 illustrates the key steps in the framework pipeline and their inputs and outputs. All algorithms in the framework were developed in Python 3.9 using packages for graph manipulation, dataset formatting, causal discovery implementation with R backends, and estimation (through high-level packages [19,25,28]). Parallelism was employed in causal discovery, estimation, and refutation using the Pebble package version 5.0.

The framework has a standardized approach to data structures, inputs, and outputs. All graphs are represented as *NetworkX* [32] graphs, a widely used graph package that can be easily translated to DOT, adjacency matrix, and other formats. This standardization ensures a consistent causal process, as the implemented discovery methods in the current causal frameworks operate uniformly while enabling easy customization and adding new methods. The graphical output is in DOT format, which can be rendered in SVG, PNG, or others.

3.1. Scope and assumptions

The framework operates within a set of foundational assumptions inherited from the CD algorithms it integrates. These include the Causal Markov Condition, Faithfulness, and Causal Sufficiency [12], which are standard prerequisites in the causal discovery literature. The framework operates exclusively on observational (non-interventional) data, estimating causal effects from statistical associations rather than controlled experiments. Regarding data characteristics, the current version focuses on cross-sectional tabular datasets with continuous, discrete, or categorical variables. Time-series data and other structured formats are outside the current scope. Additionally, the framework produces DAGs as output, meaning cyclic causal structures are not supported. Discovered causal graphs should be interpreted as statistical dependencies consistent with the data and the assumptions above, not as definitive causal truths without domain expert validation.

3.2. Input and data handling

To run the entire pipeline, only a few inputs are required: the observational dataset for analysis, the type mapping of variables in the dataset, such as categorical, continuous, and discrete, and the target column representing the observed effect. *Causal-Nest* assumes the input data is processed, cleaned, and formatted. It can handle diverse problem domains and data dimensions. If data is missing, the

[1] The code and benchmarks are available on github.com/gfviegas/causal-nest.

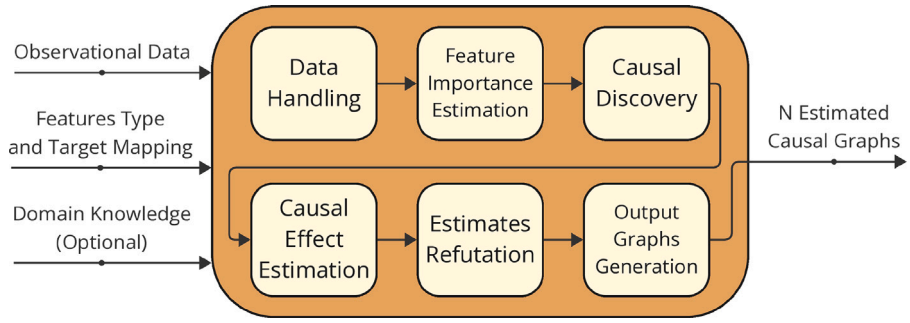


Fig. 1. Summarized pipeline of the proposed framework.

framework can drop it, apply forward fill, or interpolate based on user preference.

Although establishing causality can be challenging, some relationships, such as holidays causing rain, are known a priori to be invalid. The domain knowledge of forbidden and required edges can be provided as input, formatted in a text file respecting the syntax described in [19].

3.3. Feature importance estimation

While correlation does not directly indicate causation, it is improbable for a causal relationship to exist between two variables that are not closely connected. *Causal-Nest* includes user-defined strict time limits for tasks such as effect estimation and refutation. These tasks are ranked in a prioritized list based on their calculated feature importance, which is determined using a Random Forest classifier or regression, and automatically chosen based on the nature of the problem. Establishing this prioritized list helps prevent potentially valuable candidates for causal effects from being overlooked in later stages of the process due to time limit constraints and their lower position in the sequentially processed list of variables.

3.4. Causal discovery (CD)

Several CD algorithms are available in the literature, each with its own requirements and restrictions. Choosing the most appropriate one for a particular problem is a complex task. *Causal-Nest* includes twelve CD algorithms and automatically evaluates the ones suitable for the given problem scope, considering the data types of variables, normal distribution (as some models assume Gaussian data), and linearity. Table 2 displays the implemented CD algorithms in *Causal-Nest* along with their corresponding backends and discovery types. The framework also simplifies the implementation of other discovery methods, both in terms of execution and constraints, to be verified for running on a given dataset. It can be achieved by implementing a straightforward interface that must adhere to two functions: a Boolean function that determines whether the algorithm is suitable for a given dataset, and another function that outputs the discovered graph.

The framework validates the data type of each feature in the input dataset to ensure compatibility with the model-accepted data types. If any feature has an unsupported type, the discovery algorithm will be rejected and not applied. A D'Agostino and Pearson-based test [33] is employed for normality checks. This test assumes normality if all features yield p-values below the default threshold of 0.05, assuming the null hypothesis that the feature values follow a normal distribution. Lastly, the linearity between each variable and the target is assessed using linear regression, with the expectation that the mean residual will be below the default threshold of 0.05.

The framework uses a diverse range of CD algorithms to construct a causal graph from the provided data. This process is parallelized and allows the user to specify the maximum number of workers, which defaults to the number of CPUs in the runtime system. The framework does not prioritize the execution order of algorithms, as the choice depends primarily on the problem domain context. Each worker is responsible for creating the causal graph, generating essential statistics and rankings, and has a user-defined execution timeout. The automatic algorithm selection, parallel discovery, execution-time limits, and user-friendly statistics are fundamental features of the proposed framework, enabling systems that generate real-time data to extract causal knowledge.

Users may also specify a subset of CD algorithms to execute rather than running all applicable methods. This flexibility allows domain experts who have prior knowledge about which algorithms are most suitable for their data characteristics to focus computational resources accordingly. However, the framework does not currently implement automatic selection of the "best" CD method based on data properties. Instead, it provides comprehensive metrics (as shown in Section 4) to support expert judgment in comparing and selecting among the discovered graphs. Automatic CD algorithm recommendation based on dataset characteristics remains a direction for future research. Some methods used to derive a causality model from data were not initially designed for causality but for Bayesian networks or other use cases. As a result, the outcome of the discovery algorithms may not always produce a Directed Acyclic Graph (DAG). In such cases, *Causal-Nest* incorporates a custom-defined function to handle these situations. This function is featured in Algorithm 1 and is an adapted version of a function with a similar purpose included in the Causal Discovery Toolbox package [28] with the additional feature of handling unweighted graphs. Therefore, every CD graph generated in *Causal-Nest* is a DAG.

Algorithm 1: Implemented graph to DAG transformation

```
   Input: Graph G
   Output: Directed Acyclic Graph (DAG)
1  while ¬ G.is_directed_acyclic_graph() do
2      cycle ← G.get_next_simple_cycle()
3      if |cycle| = 2 then
4          (source_node, target_node) ← (cycle[0], cycle[1])
5          G.remove_edge(source_node, target_node)
6      end
7      else
8          (i, j) ← get_edge_with_min_weight(cycle)
9          G.invert_edge(i, j)
10     end
11 end
12 return G
```

Algorithm 1 iteratively detects and eliminates cycles until the graph becomes acyclic. The main loop (line 1) continues while cycles exist, retrieving the next cycle in line 2. The procedure distinguishes between two cases based on cycle length (line 3). For two-node cycles (bidirectional edges between a pair of nodes), line 5 simply removes one edge,

Table 2
Causal Discovery algorithms implemented by *Causal-Nest*.

CD algorithm	Type	Backend
1. Grow-Shrink (GS) [9]	Constraint-based	bnlearn [34]
2. Incremental Association Markov Blanket (IAMB) [10]	Constraint-based	bnlearn [34]
3. Causal Additive Models (CAM) [35]	Constraint-based	CAM R [35]
4. Concave Penalized Coordinate Descent with Reparametrization (CCDr) [36]	Score-based	sparsebn [37]
5. Causal Generative Neural Networks (CGNN) [38]	Score-based	pytorch [39]
6. Greedy Equivalence Search (GES) [40]	Score-based	pcalg [41]
7. Greedy Interventional Equivalence Search (GIES) [40]	Score-based	pcalg [41]
8. Linear Non-Gaussian Acyclic Model (LiNGAM) [42]	Score-based	pcalg [41]
9. Peter-Clark (PC) [11]	Constraint-based	pcalg [41]
10. Structural Agnostic Model (SAM) [43]	Score-based	pytorch [39]
11. Greedy Relaxation of the Sparsest Permutation (GRaSP) [44]	Permutation-based	causallearn [19]
12. BIC Exact Search (BES) [45]	Score-based	causallearn [19]

since reversing would recreate the same cycle. For longer cycles (lines 6–8), the algorithm identifies the edge with the minimum weight (line 7, defaulting to 1 for unweighted edges) and inverts its direction (line 8). This heuristic prioritizes preserving stronger edges while breaking cycles through weaker connections. If inversion reduces the total cycle count, the change is kept. Otherwise, the edge is removed entirely. Line 9 returns the resulting graph once no cycles remain, guaranteeing a valid DAG output regardless of the input structure.

The statistics generated in the CD step include the Forbidden Edges Violation Rate (FEVR), the Required Edges Compliance Rate (RECR), and the Knowledge Integrity Score (KIS), all of which are also contributions of this work. FEVR ratio is the percentage of forbidden edges present in the generated graph, while RECR indicates the percentage of required edges present in the graph. KIS, a score ranging from 0 to 1, is calculated using Eq. (1), which considers FEVR and RECR. A score of 1 indicates that the graph fully adheres to the knowledge restrictions, while a score of 0 indicates that the graph does not adhere to any imposed restrictions. If no knowledge input is provided, FEVR is set to 0 and RECR to 1, yielding a KIS of 1, indicating that no restrictions are violated.

$$KIS = (1 - FEVR) \times RECR \tag{1}$$

The rationale for KIS stems from a practical limitation in causal discovery benchmarking: while ground-truth causal graphs would be ideal for evaluation, they are rarely available for real-world datasets. However, domain experts can often specify partial knowledge in the form of forbidden edges (relationships known to be causally impossible) and required edges (relationships known to exist). This partial specification format, standardized by the causal-learn library [19] and adopted by benchmark repositories such as the CMU Causal Learning datasets, provides a pragmatic middle ground between complete ground truth and no validation at all.

The multiplicative formulation of KIS ensures that both constraint types must be satisfied for a high score: a graph that perfectly includes all required edges but contains many forbidden edges will be penalized, and vice versa. This design reflects the principle that domain knowledge violations in either direction indicate potential discovery failures. Unlike metrics such as SHD or SID that require a complete ground-truth graph, KIS enables quantitative evaluation using only the partial knowledge that practitioners can realistically provide.

3.4.1. Priority score

Another metric proposed in this work, assigned to a Causal Discovery graph output, is the Priority Score, as detailed in Algorithm 2. It systematically organizes graphs generated by multiple CD algorithms based on their potential to contain pivotal causal connections, with higher values indicating a greater likelihood of containing the expected causal paths. It considers multiple factors:

- **Edge Score:** Number of edges directed toward the target variable, affecting the probability of essential causal paths.
- **Distance Score:** Average shortest path distance to the target, with shorter distances indicating more direct causal influence.
- **Density Score:** Graph density, indicating greater interrelation among variables and more connected confounders.
- **Connectivity Score:** Graph connectivity, as disconnected graphs may separate important nodes from the target.
- **Betweenness Score:** Cumulative betweenness centrality of all nodes, denoting node significance in the network.

Algorithm 2: Causal Discovery generated graph priority score

Input : Graph G, target node t
Output: Ranking score
1 **if** $t \notin G.nodes()$ **then return** 0
2 $num_incoming_edges \leftarrow G$.in_degree($t$)
3 $num_nodes \leftarrow |G$.nodes()$|$
4 $edge_score \leftarrow \frac{num_incoming_edges}{num_nodes}$
5 $distances \leftarrow G$.get_nodes_shortest_distance_to(target=t)
6 $avg_distance \leftarrow \frac{\sum_{d \in distances} d}{|distances|}$
7 $distance_score \leftarrow \frac{1}{avg_distance+1}$
8 $density_score \leftarrow G$.density()
9 $connectivity_score \leftarrow 1$ if G.is_weakly_connected() else 0
10 $bc \leftarrow G$.betweenness_centrality_values()
11 $betweenness_score \leftarrow \sum_{b \in bc} b$
12 $score \leftarrow 100 \times (edge_score \times distance_score \times density_score \times connectivity_score \times betweenness_score)$
13 **return** $score$

Algorithm 2 computes the Priority Score through the following steps. Line 1 checks whether the target node exists in the graph, returning zero if absent since no causal paths to the target can exist. Next, it computes five component scores. Lines 2–4 calculate the *edge score* as the ratio of incoming edges to the target over total nodes, measuring how many variables directly influence the target. Lines 5–7 compute the *distance score* by calculating the average shortest path length from all nodes to the target using graph-theoretic distance (number of edges in the shortest path), then inverting it so that closer nodes yield higher scores. Line 8 obtains the *density score* as the standard graph density measure (ratio of actual edges to possible edges), indicating overall interconnectedness. Line 9 assigns the *connectivity score* as binary: 1 if the graph is weakly connected, 0 otherwise, penalizing graphs where some nodes are unreachable from others. Lines 10–11 compute the *betweenness score* by summing the betweenness centrality of all nodes, capturing how many shortest paths pass through each node. Finally, line 12 multiplies the five scores together and scales by 100 to produce the final Priority Score, returned in line 13.

The Priority Score is designed as a practical heuristic for execution scheduling rather than a standalone quality metric. When the framework operates under time constraints with multiple discovered graphs, it must decide which graphs to prioritize for the computationally expensive estimation and refutation phases. The score combines well-established graph-theoretic measures, namely in-degree, shortest path distance, density, connectivity, and betweenness centrality, that collectively indicate the structural richness of a graph relative to the target variable. Graphs with more paths to the target, higher connectivity, and greater node importance are processed first, maximizing the likelihood that the most informative causal relationships are evaluated before timeouts occur. The multiplicative combination ensures that graphs deficient in any single property (e.g.,disconnected graphs) receive appropriately low priority. This score is not intended as a definitive measure of graph correctness; rather, it serves the operational goal of intelligent resource allocation in time-bounded causal analysis.

3.4.2. Graphs comparison

The generated graphs can be compared using various metrics against a ground-truth graph (if available) or against each other. Within *Causal-Nest*, we have implemented three essential metrics for graph comparison: **Area Under the Precision-Recall Curve (AUCPRC)** [46]: Assesses overall classification accuracy and misclassification when comparing adjacency matrices of graphs. Higher values indicate better edge prediction; **Structural Hamming Distance (SHD)** [47]: Quantifies structural discrepancies by evaluating missing edges, extra edges, and directionality errors. Lower values indicate greater structural similarity; **Structural Intervention Distance (SID)** [48]: Examines causal relationships by computing intervention paths between all pairs of variables to confirm consistency with causal features. Lower values indicate greater causal similarity.

3.5. Causal effect estimation

Causal-Nest builds upon the DoWhy package [25] pipeline for estimating the causal effect of a variable. It is accomplished by first identifying the causal expression by assessing the graph structure and then estimating the variable effect. The proposed framework utilizes DoWhy default implementation to extract the identified estimand while disregarding unobserved confounders. Additionally, it employs DoWhy estimation methods, which can be parameterized by the user, defaulting to propensity score stratification. Therefore, the whole variable estimation process takes a causal graph and a variable as input and produces the causal estimate along with the p-value, indicating the statistical significance of the estimated causal effect.

The estimation process must be individually assessed for each variable acting as the treatment, which can lead to increased complexity. For a dataset with N independent variables and K discovered graphs, a total of $N \times K$ estimations will be carried out. In this scenario, the priority score of the discovered graphs (refer to Section 3.4) and the importance of features (refer to Section 3.3) play a crucial role in prioritizing the execution of the estimations. This prioritization is crucial as there is a hard time limit in place, which may prevent certain variables or graphs from being assessed. The estimation execution of a list of discovered graphs follows these steps:

I. Prioritize the graphs and features by sorting them in descending order based on their priority scores.
II. Create a parallelized process queue with a user-configurable maximum number of workers. By default, this maximum is set to the available CPU count in the running system.
III. Each process estimates the features sequentially, following the priority order. It has a user-configurable time limit to estimate the effect of as many variables as possible. If time runs out, partial estimations are allowed, meaning a graph can estimate a fraction of the variables.
IV. After completing a process, store the estimation results independently for each model. Then, select the following graph in the queue for estimation.

3.6. Estimates refutation

Refutation methods are built on top of the DoWhy package [25] and can easily support custom methods if they adhere to a simple interface. These methods take the dataset (and its feature mapping) and an estimation result as input and return an instance with the newly estimated effect and a p-value associated with the significance test of the newly calculated effect and the previous one. Our framework deems a refutation to have passed the refutation test if the p-value exceeds the default threshold of 0.05.

In the refutation phase, hard time limit constraints are applied due to the extensive input. Each variable serving as a treatment needs individual evaluation, and refutation must be handled for each estimation result. Multiple refutation methods are executed for each estimation result, resulting in a significant input size of $N \times K \times Z$, where N is the number of independent variables, K is the number of discovered causal graphs, and Z is the number of refuters. The proposed framework establishes a global timeout for the total execution time of the refuting pipeline and a model timeout for each estimation result entry, providing flexibility in handling different estimation results and refutation methods.

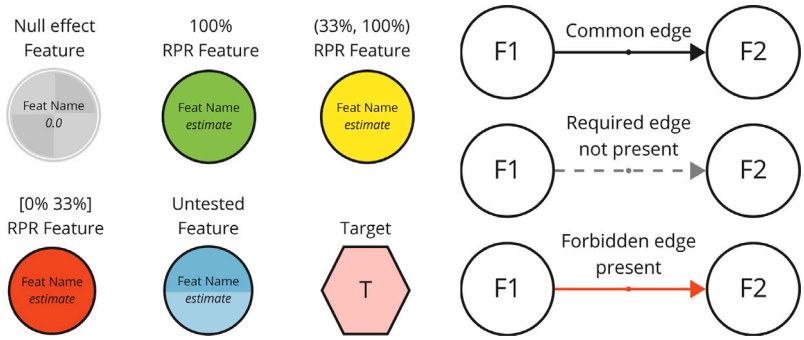
This study uses three methods of refutation: Placebo Treatment Refuter (PTR), Random Common Cause Refuter (RCCR), and Subset Removal Refuter (SRR). Each method tests different aspects of the estimated effect independently. Time constraints may limit the number of refutation tests that can be performed. The refutation execution of a list of estimation results follows these steps:

I. Prioritize the estimation results by arranging them in descending order based on their estimated effect values.
II. Create a parallelized process queue with a user-configurable maximum number of workers. By default, the maximum is set to the available CPU count.
III. Assign the process queue with independent estimation results based on the priority order. Each process sequentially tests the estimation result using the implemented refutation methods.
IV. After completing the refutations for an estimated result or reaching the model-timeout threshold, store the refutation results independently for each estimation result. Then, select the next entry in the sorted list if the global timeout has not been reached.

3.7. Output causal graphs generation

The proposed framework provides a user-friendly output for quickly evaluating the generated graphs, including their estimation and refutations, as shown in Fig. 2. It generates a custom DOT output graph for each causal graph discovered, allowing the user to inspect the causal effect estimates, the Refutation Pass Rate (RPR), and the presence of forbidden edges or the absence of required edges. Features include the feature name and the causal estimate below, while the target node is represented as a pink hexagon. Feature nodes follow a color scheme: green for variables that passed all refutation tests, red for those that passed up to 33% of the refutation tests, yellow for values in between, and split blue for variables with no refutation tests executed. A grey node appears when the estimated effect of the node is 0, indicating that the variable has no estimated causal effect on the target.

If domain knowledge is provided as input to the framework, it may display edges in various styles in addition to the standard filled black arrow, which represents a causal relationship. A filled red arrow denotes an undesired edge, as specified in the domain input. In contrast, a gray dashed edge indicates edges listed as required in the input that were not discovered by the causal discovery algorithm. In the case of a pair of nodes with both a red and a dashed gray edge, this signifies a wrongly directed edge. *Causal-Nest* can correct these edges by modifying the graphs generated during the causal discovery process with the help of an optional flag that is turned off by default.



(a) Nodes notation for output graph. (b) Edges notation for output graph.

Fig. 2. Custom nodes and edges style specification for the output graph.

4. Experimental results

This section presents the experimental evaluation of *Causal-Nest*. The evaluation is organized around two complementary scenarios:

- **Scenario 1 (Ground Truth):** Validation against the Sachs dataset [49], which has a known causal ground truth, enabling direct structural comparison using AUCPRC, SHD, and SID metrics, displayed in Section 4.1.
- **Scenario 2 (Domain Knowledge):** Evaluation on seven datasets with expert-specified forbidden and required edges, assessing the KIS metric and causal effect estimation, displayed in Section 4.2.

The primary hypothesis is that *Causal-Nest* can successfully execute the complete causal pipeline (discovery, estimation, refutation) across diverse domains, producing interpretable metrics that enable expert comparison of multiple CD algorithms. The datasets span domains including biology, aeronautics, environmental science, and engineering, with sizes ranging from 46 to 7466 instances and dimensionality from 6 to 12 features.

The experiments were conducted on a 2023 MacBook Pro with an Apple M3 Pro chip and 36 GB of memory. All applied benchmarks used identical parallelization parameters and time limits. Expressly, the maximum number of workers was set to 12. Additionally, a 30-minute timeout was established for each discovery model, for both causal discovery and estimation steps. A 30-minute global timeout and a 4-minute timeout per estimation result were applied for refutation.

4.1. Ground truth benchmark

The dataset created by Sachs et al. [49] is a widely used benchmark in causality due to its known ground truth graph, which consists of 19 directed edges. The dataset contains empirical quantitative data with 11 features and 7466 instances.

We treat the 'pjnk' column as the target and map all other columns as continuous features. The dataset does not contain missing data. After the input is given, the feature sorting is applied with the calculated associated importance from the Random Forest regressor. Table 3 shows the list of sorted features and their associated importance scores.

Fig. 4-a shows the ground-truth graph for the Sachs dataset. Although the feature importance ranking is determined by a regressor using the data rather than the ground-truth graph, two edges are pointing to the target node in the ground truth: PKC and PKA. They rank among the top 4 in the sorted list, with 0.63 and 0.04 as their priority scores, respectively. Out of the 8 paths to the target, 7 include the PKC feature, and *Causal-Nest* correctly identified its importance. Although PKA is only present in the PKA→pjnk path, it is ranked high on the list. Additionally, 4 out of the bottom 5 features in the list do not have a path to the target.

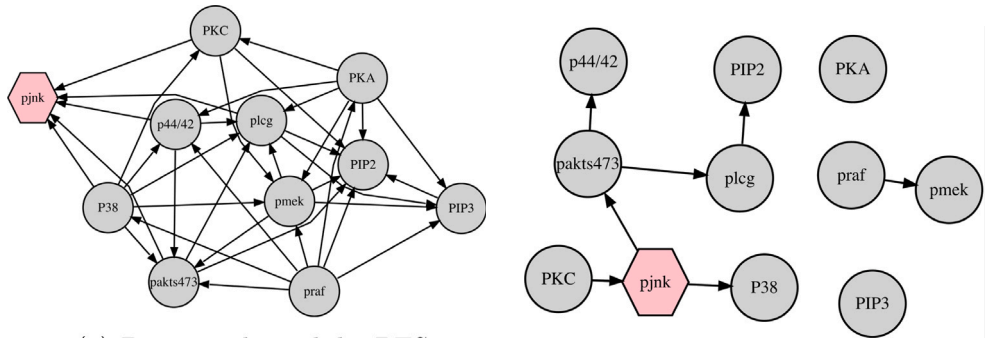
Table 3
Feature importance ranking for Sachs dataset.

	Feature	Importance score
1.	PKC	0.629
2.	plcg	0.104
3.	P38	0.088
4.	PKA	0.040
5.	PIP3	0.030
6.	pakts473	0.026
7.	p44/42	0.024
8.	praf	0.021
9.	PIP2	0.021
10.	pmek	0.017

Of the twelve discovery methods implemented, *Causal-Nest* identified eight applicable ones: PC, GS, CCDr, IAMB, SAM, BES, GRaSP, and CGNN. Due to the dataset size and complexity, SAM and CGNN timed out, while the others yielded DAGs with various calculated priority scores. BES had the highest score at 1.04, followed by IAMB (0.79), GS (0.76), PC (0.63), GRaSP (0.56), and CCDr (0.00). Fig. 3 displays the discovered graphs with the highest (a) and lowest (b) priority scores. As expected, the graph generated by BES, containing multiple paths to the target and high interconnectivity, received a higher ranking. Conversely, the final graph in the list, lacking edges pointing towards the target and disconnected, justifies its lower position in the sorting order.

Table 4 compares the metrics for the discovered graphs. Additionally, Fig. 4 shows the dataset ground truth alongside the output generated by the graph with the higher estimated causal effect (generated by PC), depicting the estimated effects of the features and their associated refutations. Notably, algorithms such as CCDr and PC demonstrated competitive performance, with AUCPRC scores of 0.45 and 0.38, respectively, indicating robust predictive power in identifying causal relationships. Achieving a 100% RPR across all algorithms highlights the capability of our solution to effectively generate robust causal models, enhancing the reliability of inferred causal relationships. The distances (SHD and SID) achieved in the benchmarks demonstrate varying levels of structural accuracy across algorithms, reflecting the diverse nature of Causal Discovery models and indicating the utility of a diverse set of algorithms in causality problems. The graph with the highest estimated impact did not closely match the ground truth metrics, as it included edges that were not present in the ground truth graph. CCDr, which had fewer edges, had fewer errors but could not accurately estimate the effect when the target was one of its disconnected nodes.

The results in Table 4 illustrate that identifying a single "best" CD method is context-dependent. When ground truth is available, AUCPRC, SHD, and SID provide direct structural comparison, where CCDr excels

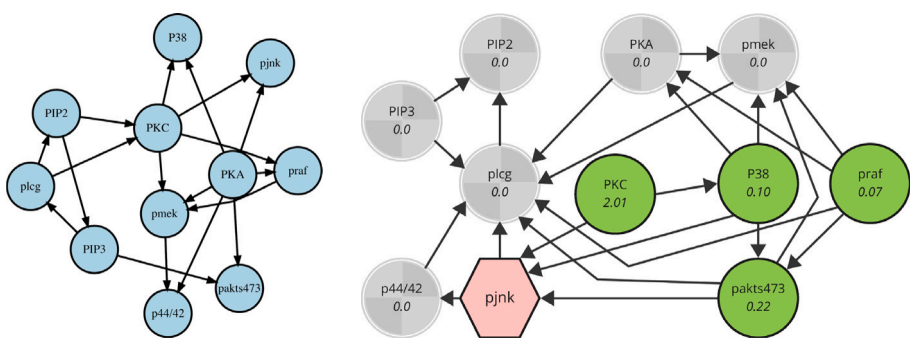


(a) Discovered graph by BES. (b) Discovered graph by CCDr.

Fig. 3. Discovered graphs sample for Sachs dataset.

Table 4
Stats for the causality pipeline on the Sachs dataset.

CD model	Priority score	AUCPRC	SHD	SID	Max causal effect estimate	Max causal effect p-value	Max causal effect RPR
1. GS	0.38	0.42	27	78	1.48	1.94e–05	100%
2. IAMB	0.77	0.25	35	90	1.83	2.57e–26	100%
4. CCDr	0.00	**0.45**	**17**	**72**	1.89	1.13e–18	100%
9. PC	0.63	0.38	28	83	**2.01**	1.50e–80	100%
11. GRaSP	0.18	0.09	42	86	1.87	3.98e–130	100%
12. BES	**1.04**	0.30	42	79	1.37	8.39e–114	100%



(a) Sachs ground truth. (b) *Causal-Nest* output for PC in Sachs dataset.

Fig. 4. Comparison between Sachs ground truth and a generated output graph.

in this example. When the goal is causal effect estimation, the maximum causal effect and RPR become more relevant, favoring PC. For exploratory analysis seeking rich causal pathways, the Priority Score and graph connectivity matter, where BES leads. This multi-metric approach is intentional: rather than imposing a single summary score, *Causal-Nest* provides diverse metrics that allow experts to select graphs aligned with their specific objectives. A composite summary metric to guide non-expert users remains a direction for future work.

4.2. Restricted benchmarks

To evaluate the performance of *Causal-Nest* across diverse real-world scenarios, we curated seven datasets from the *causal-learn* database [50], spanning multiple scientific and engineering domains. Each dataset includes domain-knowledge constraints specifying forbidden and required edges based on scientific understanding, enabling the evaluation of the Knowledge Integrity Score (KIS) and related metrics. These datasets vary significantly in domain, size (46 to 7466 instances), dimensionality (6 to 12 features), complexity, and distributional properties, providing a comprehensive evaluation of the *Causal-Nest* versatility. The benchmarked datasets are the following:

I. **Airfoil Self-Noise** [51]: Aeronautical physics dataset (1503 instances, 6 features) measuring NASA airfoil noise with features including frequency, angle of attack, chord length, and velocity.
II. **Algerian Forest Fires** [52]: Geographic/environmental dataset (244 instances, 12 features) containing meteorological data and fire indices from two Algerian regions.
III. **Yacht Hydrodynamics** [53]: Nautical physics dataset (308 instances, 7 features) predicting yacht residuary resistance from geometric and physical parameters.
IV. **Wine Quality** [54]: Food engineering dataset (1599 instances, 12 features) relating physicochemical properties to wine quality ratings.
V. **Auto MPG** [55]: Automotive engineering dataset (398 instances, 8 features) predicting fuel consumption from vehicle characteristics.
VI. **Apple Watch & Fitbit** [56]: Physical activity dataset (46 instances, 11 features) comparing wearable device measurements using indirect calorimetry as criterion.
VII. **Abalone** [57]: Marine biology dataset (4177 instances, 9 features) predicting abalone age from physical measurements.

Table 5
Datasets benchmark results: Knowledge Integrity Score metrics.

Dataset	Avg. FEVR	Avg. RECR	Avg. KIS
I.	34.00% ± 19.27%	100.00% ± 0.00%	0.66 ± 0.19
II.	0.00% ± 0.00%	100.00% ± 0.00%	1.00 ± 0.00
III.	10.00% ± 8.16%	83.33% ± 40.82%	0.77 ± 0.38
IV.	20.34% ± 11.96%	35.71% ± 47.56%	0.27 ± 0.35
V.	25.00% ± 20.41%	100.00% ± 0.00%	0.75 ± 0.21
VI.	8.08% ± 8.87%	100.00% ± 0.00%	0.82 ± 0.09
VII.	0.00% ± 0.00%	100.00% ± 0.00%	1.00 ± 0.00

Table 6
Datasets benchmark results: Maximum causal estimate metrics.

Dataset	Avg. Max causal Est.	Avg. Max p-value	Avg. Max RPR
I.	0.15 ± 0.08	3.91e–06 ± 3.39e–06	100.00% ± 0.00%
II.	0.03 ± 0.04	8.37e–11 ± 1.98e–10	75.00% ± 46.29%
III.	101.42 ± 49.59	1.23e–01 ± 3.00e–01	100.00% ± 0.00%
IV.	18.58 ± 45.53	1.94e–02 ± 5.14e–02	100.00% ± 0.00%
V.	0.63 ± 0.46	7.13e–04 ± 1.43e–03	100.00% ± 0.00%
VI.	0.06 ± 0.00	4.04e–02 ± 0.00e–01	100.00% ± 0.00%
VII.	28.76 ± 20.10	5.65e–04 ± 7.99e–04	100.00% ± 0.00%

The entire pipeline was executed for each dataset, encompassing all stages from data management to refutations. Although these datasets lack ground truth for comparing graph similarities, they do offer knowledge information about forbidden and required edges. Consequently, we computed the FEVR, RECR, and KIS for each discovery method employed to evaluate how well the graphs adhered to the provided knowledge. Additionally, we calculated the maximum causal estimate, its corresponding p-value, and the RPR.

The benchmark results, presented in Tables 5 and 6, demonstrate the utility of the *Causal-Nest* framework for robust causal analysis across heterogeneous domains. A primary strength highlighted by these benchmarks is the successful application to a wide array of fields, from physics and biology to engineering, confirming its domain-agnostic capabilities. Strong performance across multiple datasets demonstrates the ability of the framework to produce structurally sound and statistically reliable models. For instance, as shown in Table 5, *Dataset I* and *Dataset VII* achieved perfect KIS scores (1.00) with 0.00% FEVRs and 100.00% RECRs, indicating that the discovery algorithms consistently converged on graphs that were perfectly aligned with established domain knowledge. Other datasets, such as *Dataset III* (0.77 KIS) and *Dataset VI* (0.82 KIS), also showed robust knowledge alignment.

Beyond simple validation, the actual value of *Causal-Nest* lies in its diagnostic power, which is essential for real-world applications. Rather than merely failing, the framework pinpoints specific challenges within problematic datasets. The *Dataset IV*, for example, serves as a critical outlier. Its extremely low KIS (0.27 ± 0.35) is driven by a critically low average RECRs (35.71% ± 47.56%), as detailed in Table 5. This finding is highly instructive, as it suggests the constituent discovery algorithms are not just misdirecting edges but are fundamentally failing to identify known required relationships in this specific data-generating process. Similarly, the high standard deviations for FEVRs in *Dataset I* (±19.27%) and *Dataset V* (±20.41%) highlight instability and disagreement among the underlying discovery methods when applied to these data.

This diagnostic strength extends to the causal estimate analysis. The multi-metric evaluation (KIS, p-value, RPRs) provides a necessary comprehensive assessment. A prime example is *Dataset III*, where a high KIS (0.77, Table 5) is contrasted with a non-significant average p-value of *0.123* (± 0.30) for its maximum causal effect (Table 6). *Causal-Nest* thus correctly flags that while the graph structure is plausible, the key causal estimate is not statistically reliable. Furthermore, the high RPRs variability in *Dataset II* (75.00% ± 46.29%, Table 6), despite a perfect KIS (Table 5) and significant p-value (Table 6), demonstrates the ability of the framework to identify estimates that are not robust to refutation.

In summary, these benchmarks validate *Causal-Nest* as a powerful exploratory and diagnostic tool. Its successful deployment across diverse scientific fields, coupled with its ability to precisely identify structural failures (*Dataset IV*, Table 5), statistical insignificance (*Dataset III*, Table 6), and refutation instability (*Dataset II*, Table 6), underscores its utility for navigating the complex, non-uniform challenges of real-world causal discovery.

4.3. From validation to practical application

Having established the reliability of the framework in these controlled, knowledge-rich scenarios, we now shift our focus to a more common and practical application. In many real-world machine learning problems, a causal ground truth is not available. The primary goal is not validation, but improving a predictive outcome. The following section presents a case study to evaluate *Causal-Nest* as a practical tool for causality-driven feature engineering.

5. Real-world application: Causality-driven feature engineering for churn prediction

This section presents a case study applying *Causal-Nest* to a practical machine learning problem where no ground truth causal graph exists. The evaluation is structured as follows:

- **Research Question:** Can causality-driven feature engineering, guided by *Causal-Nest*, improve predictive performance for customer churn compared to baseline features?
- **Dataset:** Iranian Telecom customer churn dataset [58] with demographic and behavioral features.
- **Methodology:** Compare five feature sets (Baseline, Causal, Baseline OS, Causal OS, Causal Univariate) across three ML classifiers (CBC, GBMC, DTC).
- **Metrics:** Class-1 F1-Score, Precision, and Recall for the minority churn class. Graph structure metrics (PTT, Edges in PTT, Edges-to-Nodes Ratio) for causal graph quality.
- **Expected Outcome:** Causally-engineered features should produce richer causal graph structures and potentially improve minority class prediction.

To demonstrate the utility of the *Causal-Nest* framework in a real-world predictive modeling scenario, we present a case study on customer churn prediction in the telecommunications industry. This application highlights how the framework can be leveraged to move beyond correlation-based prediction toward more robust, causality-informed feature engineering. The study investigates the application of causal discovery, facilitated by *Causal-Nest*, to enhance feature engineering in Machine Learning (ML) models predicting customer churn. By generating causal graphs from observational data and deriving feature sets informed by these structures, the framework may improve the predictive performance, particularly for the minority churn class, compared to the baseline raw data.

It is important to note that the experimental comparison in this section evaluates different *feature engineering strategies* rather than comparing *Causal-Nest* against other causal frameworks. The baseline represents a traditional ML approach using raw features, while the causal feature sets represent features engineered with guidance from the causal graphs discovered by *Causal-Nest*. This design isolates the contribution of causality-driven feature engineering to predictive performance.

Table 7
Derived causal features and their conceptual formulas.

Feature	Conceptual formula/Description
Complaint severity	Complaints × Call failures
Loyalty index	Subscription Length × Usage frequency
Service quality	Usage/(Call Failures + 1)
Value perception	Usage/Charge amount
Communication preference	SMS frequency/(Call Frequency + 1)
Network breadth	Distinct numbers Called/Total calls
Customer lifecycle	Customer age × Subscription length
Price sensitivity	Charge Amount/Customer value

5.1. Methodology and feature engineering

The study used a publicly available customer churn dataset [58] and employed the *Causal-Nest* framework to explore the impact of causality on feature sets systematically. The core idea was to generate causal feature sets by identifying attributes that encapsulate critical elements of customer experience and behavior hypothesized to have causal relationships with customer churn. This was achieved by using the framework pipeline to generate causal graphs, and then deriving new, causally informed features (e.g., Complaint Severity, Loyalty Index, Value Perception) that represent causal pathways or mitigate confounding effects.

The causal feature engineering process was iterative, using *Causal-Nest* as a diagnostic tool to guide feature construction. The workflow proceeded as follows: (1) Run the framework on the current feature set to generate causal graphs. (2) Analyze the graph structure metrics, specifically Paths to Target (PTT), edge connectivity, and graph density. (3) Use graph insights to hypothesize new composite features that might capture causal pathways not evident in raw variables. (4) Re-run the framework with the augmented feature set. (5) Iterate until graphs exhibit richer causal structure toward the target variable. This process produced eight key derived features, listed in Table 7, each designed to capture a hypothesized causal mechanism.[2]

Five distinct feature sets were compared:

- **Baseline:** The original, raw features.
- **Causal:** The causally-engineered features, including the newly derived variables.
- **Baseline OS & Causal OS:** Oversampled versions of the Baseline and Causal sets using SMOTE to address class imbalance.
- **Causal Univariate:** A feature selection approach on the Causal set using scikit-learn SelectKBest with ANOVA F-value (f_classif), retaining the top 10 features with the strongest statistical relationship to Churn. This simulates a common practice where a data engineer without causality expertise applies standard univariate feature selection.

5.2. Causal graph analysis

The framework executed several CD algorithms on each feature set: Peter-Clark (PC), Grow-Shrink (GS), Concave Penalized Coordinate Descent with Reparametrization (CCDr), Incremental Association Markov Blanket (IAMB), Structural Agnostic Model (SAM), BIC Exact Search (BES), and Greedy Relaxation of the Sparsest Permutation (GRaSP). Each method was configured with a uniform timeout of two hours. Due to this time constraint, not all algorithms were completed for every feature set. For the Baseline and Baseline OS sets, algorithms that finished include BES, CCDr, GRaSP, GS, IAMB, and PC. In contrast, for the Causal sets, only CCDr and PC succeeded due to higher dimensionality, resulting in timeouts for the others.

After the CD step, *Causal-Nest* proceeded to causal estimation and refutation for all identified graphs without timeouts. This pipeline facilitated feature-set comparison through a causal lens and the generation of causal feature sets in prior executions. We also assessed the structural relevance of the generated graphs to churn prediction using standard graph metrics from network analysis libraries, listed below:

- **Paths to Target (PTT):** The count of directed paths originating from other feature nodes and terminating at the *Churn* node. Generally, a greater number of identified paths indicates a higher probability that the discovery algorithm has revealed a significant causal connection. Subsequently, these paths are rigorously evaluated during causal estimation and refutation and can therefore be filtered by domain specialists.
- **Edges in Paths to Target:** The total number of unique edges participating in these directed paths. This metric quantifies the direct and indirect structural connectivity leading to the target variable within the learned graph, which we hypothesize correlates with the predictive utility of the feature set.
- **Edges-to-Nodes Ratio**: The ratio of edges to nodes in a graph, an essential measure for assessing the connectivity of generated graphs. A higher ratio can indicate richer interconnectivity, which may be beneficial in complex domains. However, there is no universal optimal value, as its significance varies across datasets and contexts. This metric also enables comparing graphs generated by the same causal discovery algorithm across different feature sets, providing insight into how feature composition influences the structure of learned causal relationships.

Table 8 presents a summary of the findings from the extracted metrics of causal graphs, indicating a notable enrichment of causal structures achieved by employing causally engineered feature sets.

One significant observation is that algorithms such as GRaSP, IAMB, and GS struggled to identify complex causal relationships under time constraints. This resulted in graphs with a limited number of Paths to Target (PTT) and Edges in PTT, thereby limiting their causal insights, regardless of the feature set used.

In contrast, the PC and CCDr algorithms demonstrated a more comprehensive discovery of causal structures, particularly when using the Causal feature set. For instance, the PC algorithm achieved 40 PTT and 172 Edges in PTT, significantly exceeding baseline results. The CCDr algorithm achieved impressive metrics — 882 PTT and 6647 Edges in PTT — suggesting that the Causal feature set effectively uncovers intricate causal networks. Although the Causal OS for CCDr produced no paths, the Causal Univariate still showed strong potential, yielding 345 PTT and 1509 Edges.

Meanwhile, the BES algorithm, despite generating a substantial number of paths with the Baseline feature set, demonstrated a lack of causal strength in its connections. The Edges-to-Nodes Ratio revealed that the Causal and Causal OS feature sets for both PC and CCDr often exhibit higher density, particularly in targeting pathways. Notably, the Causal feature set for CCDr achieved an Edges-to-Nodes Ratio of 3.13.

Overall, the graphs derived from the Causal feature set, especially those influenced by CCDr, yielded 6819 edges in causal paths, underscoring the benefits of using causally engineered features to enhance causal structure in churn prediction.

Causal-Nest holds excellent promise as a valuable resource for ML engineers, offering guidance for feature engineering that leads to more impactful causal graphs, all without the need to learn and implement multiple causal discovery and estimation algorithms, as the framework manages these processes automatically.

5.3. Predictive performance

Three ML models were evaluated: Categorical Boosting Classifier (CBC), Gradient Boosting Machine Classifier (GBMC), and Decision Tree Classifier (DTC). This work replicates the setup from Hossain [59]

[2] The complete implementation is available at https://github.com/gfviegas/itcp-causal-benchmark/.

Table 8
Causal graphs metrics for each feature set.

CD Algorithm	Feature set	PTT	Edges in PTT	Edges-to-Nodes ratio
PC	Baseline	1	1	1.57
	Baseline OS	5	8	1.71
	Causal	**40**	**172**	1.47
	Causal OS	8	18	**1.89**
CCDr	Baseline	1	1	1.79
	Baseline OS	7	12	**4.00**
	Causal	**882**	**6647**	3.13
	Causal OS	0	0	0.74
	Causal Uni.	345	1509	**4.00**
BES	Baseline	**600**	**3162**	3.79
	Baseline OS	14	40	**4.36**
	Causal Uni.	24	62	3.09
GRaSP	Baseline	**1**	**1**	2.79
	Baseline OS	**1**	**1**	**3.57**
	Causal Uni.	**1**	**1**	2.55
IAMB	Baseline	1	1	**1.64**
	Baseline OS	**2**	**2**	1.57
GS	Baseline	0	0	**1.64**
	Baseline OS	**2**	**2**	1.57

and selects the models with most consistent results on the Class-1 prediction.

Fig. 5 presents Class-1 F1-Scores across all models and feature sets using an 80/20 train–test split, while Figs. 6 and 7 show corresponding precision and recall metrics. The F1 Score is a critical metric for imbalanced classification tasks such as churn prediction. Analysis reveals that the CBC model achieved the highest F1 Score of 90.55% with the Causal OS feature set, closely followed by the Baseline feature set at 90.36%. This indicates that combining causally informed features with oversampling effectively optimizes performance. The GBMC model also performed well, attaining an F1 Score of 89.30% with the Causal OS set, while its Causal feature set (without oversampling) yielded a score of 88.21%. In contrast, the DTC model consistently underperformed, with a maximum F1 Score of 82.47% using the Causal feature set, underscoring the limitations of a single decision tree compared to ensemble methods.

Examining precision and recall trends, oversampling generally improved Class-1 Recall across models, with GBMC reaching 96.97% and CBC at 92.93%, enhancing the identification of churners. However, precision values notably declined with oversampling; for instance, GBMC precision fell from 87.50% (Baseline) to 69.23% (Baseline OS), illustrating the common trade-off in imbalanced classification. Notably, the Causal feature set achieved the highest precision scores for both CBC (94.32%) and GBMC (89.58%), indicating that causally informed features can yield more accurate positive predictions.

Overall, integrating causally informed features generally improved performance for the churn class compared to the Baseline, with the highest F1 scores achieved for both CBC and GBMC using the Causal OS feature set. Additionally, the Causal Univariate set demonstrated competitive performance for the DTC, suggesting the significant predictive power of a subset of causal features.

5.4. Models robustness

The robustness of the models was evaluated using a 10-fold cross-validation approach. To maintain the purity of the validation fold and prevent any influence from oversampling techniques, these methods were exclusively applied to the training dataset. This strategic choice preserves the integrity of the validation process, allowing for an objective assessment of the performance. Table 9 displays the results, including the mean and standard deviation calculated across the folds. The results consistently demonstrated the benefits of incorporating causal insights into feature engineering:

Table 9
Mean and standard deviation metrics of cross validation.

	Feature set	(Mean ± Std)		
		Class-1 F1 Score	Class-1 Precision	Class-1 Recall
CBC	Baseline	86.88 ± 2.40	90.35 ± 2.76	83.86 ± 4.45
	Baseline OS	88.24 ± 2.87	84.75 ± 2.78	**92.17 ± 4.55**
	Causal	87.20 ± 2.82	**91.33 ± 3.52**	83.60 ± 4.35
	Causal OS	**88.86 ± 3.10**	85.98 ± 3.09	**92.17 ± 5.21**
	Causal Uni.	83.24 ± 4.18	85.76 ± 4.57	81.08 ± 5.57
DTC	Baseline	79.60 ± 5.10	**81.94 ± 5.14**	77.53 ± 5.93
	Baseline OS	80.42 ± 4.11	78.17 ± 5.96	**83.08 ± 4.07**
	Causal	79.48 ± 5.05	80.95 ± 5.77	78.31 ± 6.30
	Causal OS	**80.45 ± 5.16**	78.82 ± 6.30	82.36 ± 5.43
	Causal Uni.	76.55 ± 3.47	74.99 ± 3.84	78.85 ± 7.85
GBMC	Baseline	80.89 ± 3.55	86.65 ± 4.40	76.03 ± 4.55
	Baseline OS	79.27 ± 3.79	70.37 ± 4.40	90.92 ± 4.23
	Causal	83.27 ± 1.91	**87.32 ± 3.95**	79.81 ± 3.47
	Causal OS	**84.90 ± 3.94**	79.29 ± 4.41	**91.69 ± 5.94**
	Causal Uni.	79.55 ± 4.23	83.06 ± 5.78	76.53 ± 4.62

- **Causal OS Superiority:** The Causal Oversampled (Causal OS) feature set consistently yielded the highest Class-1 F1 Scores for the ensemble models (CBC and GBMC) in both single-split and 10-fold cross-validation. This highlights the synergistic effect of combining causally-informed features with techniques to address class imbalance.
- **Precision and Recall Trade-off:** While oversampling generally led to higher Class-1 Recall (critical for minimizing missed churners), the Causal feature set (without oversampling) yielded the highest Class-1 Precision for both CBC and GBMC. This indicates that the causally-informed features contribute to more accurate positive predictions, which is valuable for targeted retention campaigns.
- **Robustness:** The superior performance of models trained on causally derived features suggests that features representing causal pathways are more robust and informative for churn prediction, offering a significant advantage over features derived merely from correlation.

These results are consistent with the theoretical motivation for causal feature engineering discussed in the literature [4,5]. Prior work suggests that features derived from causal relationships, rather than mere correlations, may offer advantages such as reduced sensitivity to coincidental associations and better generalization under distribution shift. However, it is important to note that this case study does not provide definitive proof of these theoretical claims. Rather, it demonstrates an empirical improvement in a practical scenario. The observed performance gains, particularly the higher precision of the Causal feature set and the overall F1 improvement of Causal OS, are consistent with the hypothesis that causally-informed features capture more stable predictive signals. Future work with controlled experiments and multiple domains would be necessary to establish stronger causal claims about the benefits of this approach.

6. Conclusion

This paper has introduced a new framework, *Causal-Nest*, that significantly advances causal inference for AI model developers, particularly those with minimal expertise in causality. By integrating multiple causal discovery algorithms and addressing crucial aspects such as time constraints and prioritization, *Causal-Nest* effectively simplifies the complex process of causal analysis.

Our comprehensive experimental results demonstrate the capability of the framework to validate causal structures against known ground truths. The *Causal-Nest* performance on the Sachs [49] dataset, as well

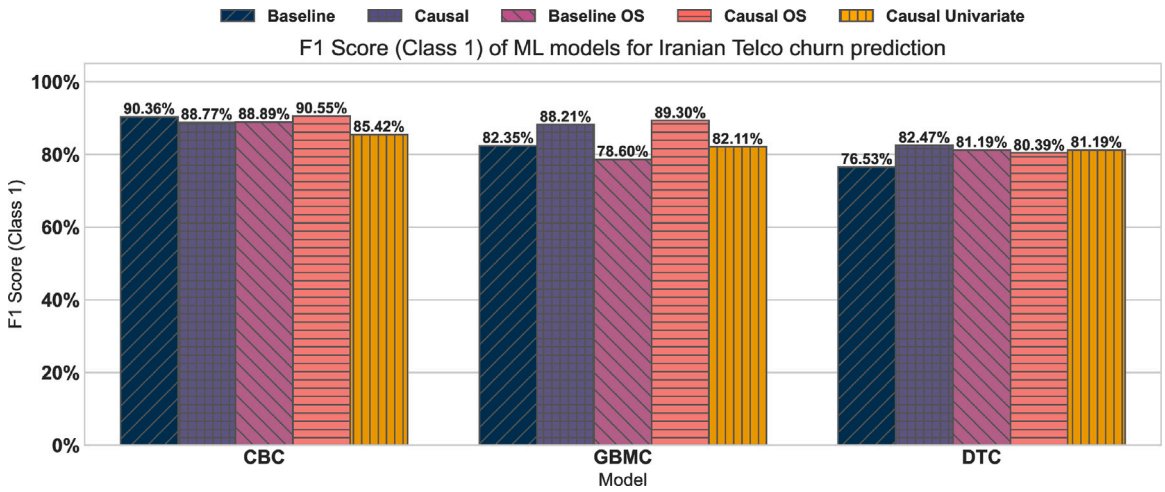


Fig. 5. F1 Score metrics for the churn class of ML models for each feature set.

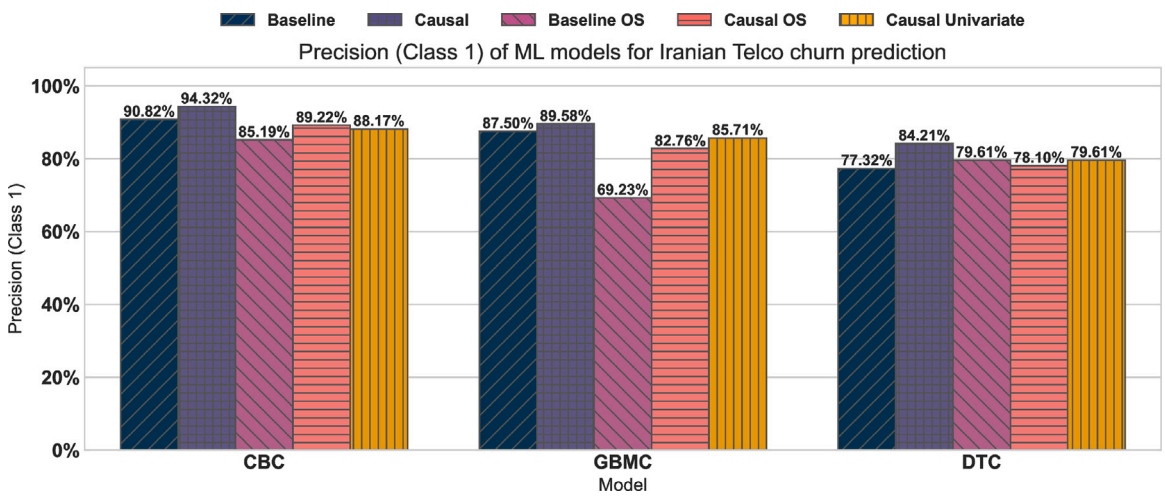


Fig. 6. Precision metrics for the churn class of ML models for each feature set.

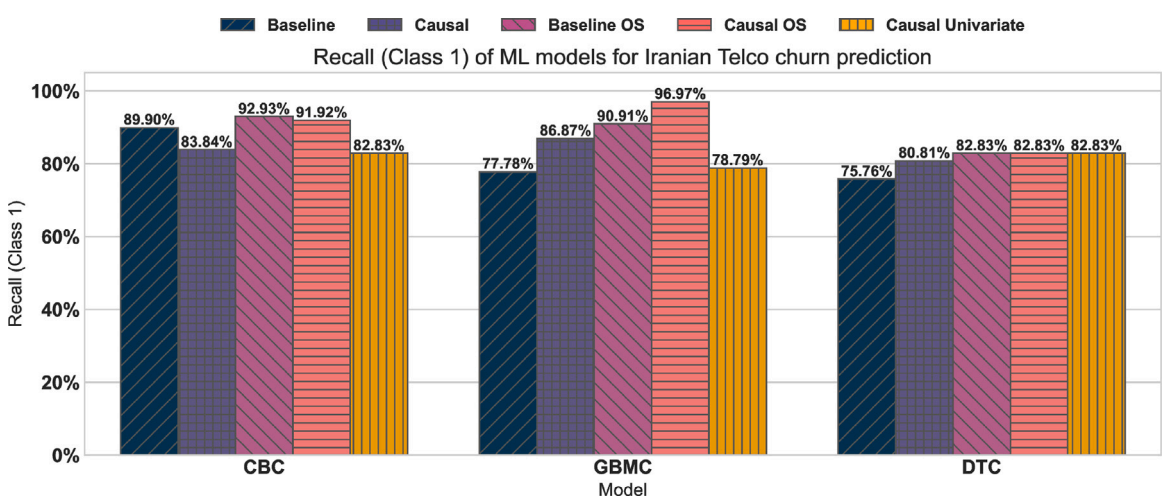


Fig. 7. Recall metrics for the churn class of ML models for each feature set.

as our extended benchmarks, demonstrates its effectiveness. It successfully identified known relationships (e.g., in the *Abalone* and *Algerian Forest Fires* datasets) while also serving as a robust diagnostic tool, flagging structural discovery failures (e.g., the low RECRs in the *Wine Quality* dataset) and statistical unreliability (e.g., the non-significant p-value in the *Nautical Physics* dataset). These findings, combined with user-friendly outputs like custom DOT graphs, make complex causal analysis accessible.

In conclusion, *Causal-Nest* bridges a critical gap in AI development, providing a single, powerful tool for both uncovering and validating causal relationships. These capabilities hold significant promise for decision-making in high-stakes fields such as healthcare, justice, and finance.

From a practitioner perspective, *Causal-Nest* directly reduces the barrier to causal analysis adoption in real-world settings. Data engineers and machine learning practitioners can leverage the framework to guide causality-driven feature engineering without requiring deep expertise in causal methodology, as demonstrated by the telecommunications churn case study. The framework is designed for direct integration into existing data science workflows, offering immediate value in time-sensitive application domains such as healthcare decision support, financial risk assessment, and customer behavior modeling, where understanding causal mechanisms rather than mere correlations can substantially improve model reliability and fairness.

6.1. Limitations

The current version of *Causal-Nest* has limitations. The framework operates exclusively on cross-sectional data and does not support time-series causal discovery methods. Output graphs are constrained to directed acyclic graphs, meaning domains with inherent feedback loops or cyclic causal structures are not supported. The framework assumes observational data, and interventional or experimental data integration is not currently implemented. Users must interpret multiple metrics to select the most appropriate CD algorithm, as a single summary metric is not provided.

6.2. Future work

Several directions for future research emerge from this work. Extending the framework to support time-series CD through integration with specialized methods such as Tigramite [26] or CausalFlow [27] would broaden applicability to domains such as finance and climate science. Meta-learning approaches could enable automatic CD algorithm recommendation based on dataset characteristics, along with a composite evaluation metric to guide non-expert users.

Of particular interest are ensemble methods that combine results from multiple CD algorithms to identify convergent structures across different graph outputs. Limited work exists in the literature on combining CD models to find overall convergence, an area that could be explored through techniques such as path relinking. When domain knowledge specifying forbidden and required edges is available, this information could guide graph optimization toward maximizing the KIS. Additionally, exploring alternative DAG transformation strategies could uncover superior causal graphs, presenting opportunities for metaheuristic methods to find optimal directed graphs from undirected structures.

From an empirical perspective, validation on larger-scale datasets and comparison with alternative causal feature selection methods such as Transfer Entropy [60] would strengthen confidence in the framework. Integration with popular ML pipelines and AutoML systems could facilitate broader adoption in industry settings. *Causal-Nest* is open-source, and the community is encouraged to contribute new CD algorithms, estimation methods, or domain-specific extensions through the project repository.

CRediT authorship contribution statement

Gustavo F.V. de Oliveira: Writing – review & editing, Writing – original draft, Validation, Supervision, Software, Methodology, Investigation, Formal analysis, Conceptualization. **Fabrício A. Silva:** Writing – review & editing, Supervision, Project administration, Methodology, Conceptualization. **Marcus H.S. Mendes:** Writing – review & editing, Supervision, Methodology.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This study was supported by CNPq and FAPEMIG.

Data availability

We shared Github repositories links in the text.

References

[1] J. Pearl, Causality, Cambridge University Press, 2009, http://dx.doi.org/10.1017/CBO9780511803161.
[2] Z. Obermeyer, B. Powers, C. Vogeli, et al., Dissecting racial bias in an algorithm used to manage the health of populations, Science 366 (6464) (2019) 447–453, http://dx.doi.org/10.1126/science.aax2342.
[3] D. Ensign, S.A. Friedler, S. Neville, et al., Runaway feedback loops in predictive policing, in: Conference on Fairness, Accountability and Transparency, PMLR, 2018, pp. 160–171.
[4] B. Schölkopf, Causality for machine learning, in: Probabilistic and Causal Inference: The Works of Judea Pearl, first ed., Association for Computing Machinery, 2022, pp. 765–804, http://dx.doi.org/10.1145/3501714.3501755.
[5] K. Yu, X. Guo, L. Liu, et al., Causality-based feature selection: Methods and evaluations, ACM Comput. Surv. 53 (5) (2020) http://dx.doi.org/10.1145/3409382.
[6] M.A. Alcorn, Q. Li, Z. Gong, et al., Strike (with) a pose: Neural networks are easily fooled by strange poses of familiar objects, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2019, pp. 4845–4854, http://dx.doi.org/10.1109/CVPR.2019.00498.
[7] H.Q. Yu, Dynamic causality knowledge graph generation for supporting the chatbot healthcare system, in: Proceedings of the Future Technologies Conference (FTC) 2020, Volume 3, Springer International Publishing, 2021, pp. 30–45, http://dx.doi.org/10.1007/978-3-030-63092-8_3.
[8] R. Moraffah, P. Sheth, M. Karami, et al., Causal inference for time series analysis: Problems, methods and evaluation, Knowl. Inf. Syst. 63 (2021) 3041–3085, http://dx.doi.org/10.1007/s10115-021-01621-0.
[9] D. Margaritis, et al., Learning Bayesian Network Model Structure from Data (Ph.D. thesis), School of Computer Science, Carnegie Mellon University Pittsburgh, PA, USA, 2003.
[10] I. Tsamardinos, C.F. Aliferis, A.R. Statnikov, et al., Algorithms for large scale Markov blanket discovery, in: FLAIRS, Vol. 2, 2003, pp. 376–381.
[11] M. Kalisch, P. Bühlman, Estimating high-dimensional directed acyclic graphs with the PC-algorithm, J. Mach. Learn. Res. 8 (3) (2007).
[12] P. Spirtes, C. Glymour, R. Scheines, Causation, Prediction, and Search, MIT Press, 2000.
[13] Y. Xia, H. Zhang, Y. Ren, J. Guan, S. Zhou, Causal discovery by continuous optimization with conditional independence constraint: Methodology and performance, in: 2023 IEEE International Conference on Data Mining, ICDM, 2023, pp. 668–677, http://dx.doi.org/10.1109/ICDM58522.2023.00076.
[14] Y. Zhu, P.V. Benos, M. Chikina, A hybrid constrained continuous optimization approach for optimal causal discovery from biological data, Bioinformatics 40 (Supplement_2) (2024) http://dx.doi.org/10.1093/bioinformatics/btae411.
[15] K. Zhou, M. Jiang, B. Gabrys, Y. Xu, Learning causal representations based on a GAE embedded autoencoder, IEEE Trans. Knowl. Data Eng. 37 (6) (2025) 3472–3484, http://dx.doi.org/10.1109/TKDE.2025.3546607.
[16] D.C. Montgomery, E.A. Peck, G.G. Vining, Introduction to Linear Regression Analysis, Wiley, 2012.
[17] J. Friedman, T. Hastie, R. Tibshirani, The Elements of Statistical Learning: Data Mining, Inference, and Prediction, Springer, 2001.
[18] G.W. Imbens, D.B. Rubin, Causal Inference for Statistics, Social, and Biomedical Sciences: An Introduction, Cambridge University Press, 2015, http://dx.doi.org/10.1017/CBO9781139025751.
[19] Y. Zheng, B. Huang, W. Chen, et al., Causal-learn: Causal discovery in Python, 2023, arXiv:2307.16405.
[20] A. Ankan, J. Textor, pgmpy: A Python Toolkit for Bayesian Networks, J. Mach. Learn. Res. (25) (2024) 1–8.
[21] M. Scutari, Learning Bayesian Networks with the bnlearn R package, J. Stat. Softw. 35 (3) (2010) 1–22, http://dx.doi.org/10.18637/jss.v035.i03, URL https://www.jstatsoft.org/index.php/jss/article/view/v035i03.

[22] J. Ramsey, B. Andrews, Py-Tetrad and RPy-Tetrad: A new Python interface with R support for tetrad causal search, in: E. Kummerfeld, S. Ma, E. Rawls, B. Andrews (Eds.), Proceedings of the 2023 Causal Analysis Workshop Series, in: Proceedings of Machine Learning Research, Vol. 223, PMLR, 2023, pp. 40–51, URL https://proceedings.mlr.press/v223/ramsey23a.html.

[23] V. Syrgkanis, G. Lewis, M. Oprescu, et al., Causal inference and machine learning in practice with EconML and CausalML: Industrial use cases at Microsoft, TripAdvisor, Uber, in: Proceedings of the 27th ACM SIGKDD, KDD '21, ACM, 2021, pp. 4072–4073, http://dx.doi.org/10.1145/3447548.3470792.

[24] H. Chen, T. Harinen, J.-Y. Lee, et al., CausalML: Python package for causal machine learning, 2020, http://dx.doi.org/10.48550/ARXIV.2002.11631.

[25] A. Sharma, E. Kiciman, DoWhy: An End-to-End library for causal inference, 2020, http://dx.doi.org/10.48550/ARXIV.2011.04216.

[26] J. Runge, P. Nowack, M. Kretschmer, S. Flaxman, D. Sejdinovic, Detecting and quantifying causal associations in large nonlinear time series datasets, Sci. Adv. 5 (11) (2019) eaau4996, http://dx.doi.org/10.1126/sciadv.aau4996.

[27] L. Castri, S. Mghames, M. Hanheide, N. Bellotto, CAnDOIT: Causal discovery with observational and interventional data from Time-Series, Adv. Intell. Syst. (2024) http://dx.doi.org/10.1002/aisy.202300188.

[28] D. Kalainathan, O. Goudet, Causal discovery toolbox: Uncover causal relationships in Python, 2019, http://dx.doi.org/10.48550/ARXIV.1903.02278.

[29] K. Zhang, S. Zhu, M. Kalander, et al., gCastle: A Python toolbox for causal discovery, 2021, arXiv preprint arXiv:2111.15155.

[30] QuantumBlack Labs, CausalNex, 2021, https://github.com/mckinsey/causalnex.

[31] D. Arpit, M. Howson, et al., Salesforce CausalAI Library: A fast and scalable framework for causal analysis of time series and tabular data, 2023, arXiv preprint arXiv:2301.10859.

[32] A. Hagberg, P.J. Swart, D.A. Schult, Exploring Network Structure, Dynamics, and Function Using Networkx, Tech. Rep., Los Alamos National Laboratory (LANL), Los Alamos, NM (United States), 2008.

[33] R. D'agostino, E.S. Pearson, Tests for departure from normality. Empirical results for the distributions of b_2 and $\sqrt{b_1}$, Biometrika 60 (3) (1973) 613–622.

[34] M. Scutari, Learning Bayesian networks with the bnlearn R package, J. Stat. Softw. 35 (3) (2010) 1–22, http://dx.doi.org/10.18637/jss.v035.i03.

[35] P. Bühlmann, J. Peters, J. Ernest, CAM: Causal additive models, high-dimensional order search and penalized regression, Ann. Statist. 42 (6) (2014) 2526–2556, URL http://www.jstor.org/stable/43556502.

[36] B. Aragam, Q. Zhou, Concave penalized estimation of sparse Gaussian Bayesian networks, J. Mach. Learn. Res. 16 (1) (2015) 2273–2328.

[37] B. Aragam, J. Gu, Q. Zhou, Learning Large-Scale Bayesian networks with the sparsebn package, J. Stat. Softw. 91 (11) (2019) http://dx.doi.org/10.18637/jss.v091.i11.

[38] O. Goudet, D. Kalainathan, P. Caillou, et al., Learning functional causal models with generative neural networks, Explain. Interpret. Model. Comput. Vis. Mach. Learn. (2018) 39–80.

[39] Y.e.a. Ansel, PyTorch 2: Faster machine learning through dynamic python bytecode transformation and graph compilation, in: Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, ASPLOS '24, Vol. 2, Association for Computing Machinery, 2024, pp. 929–947, http://dx.doi.org/10.1145/3620665.3640366.

[40] P. Nandy, A. Hauser, M.H. Maathuis, High-dimensional consistency in score-based and hybrid structure learning, Ann. Statist. 46 (6A) (2018) 3151–3183, http://dx.doi.org/10.1214/17-AOS1654.

[41] M. Kalisch, M. Mächler, D. Colombo, et al., Causal inference using graphical models with the R Package pcalg, J. Stat. Softw. 47 (11) (2012) 1–26, http://dx.doi.org/10.18637/jss.v047.i11.

[42] S. Shimizu, P.O. Hoyer, A. Hyvärinen, et al., A linear non-Gaussian acyclic model for causal discovery, J. Mach. Learn. Res. 7 (10) (2006).

[43] D. Kalainathan, O. Goudet, I. Guyon, et al., Structural agnostic modeling: Adversarial learning of causal graphs, J. Mach. Learn. Res. 23 (219) (2022) 1–62.

[44] W.-Y. Lam, B. Andrews, J. Ramsey, Greedy relaxations of the sparsest permutation algorithm, in: Uncertainty in Artificial Intelligence, PMLR, 2022, pp. 1052–1062.

[45] C. Yuan, B. Malone, Learning optimal Bayesian networks: A shortest path perspective, J. Artificial Intelligence Res. 48 (2013) 23–65, http://dx.doi.org/10.1613/jair.4039.

[46] K. Boyd, K.H. Eng, C.D. Page, Area under the Precision-Recall Curve: Point estimates and confidence intervals, in: H. Blockeel, K. Kersting, S. Nijssen, F. Železný (Eds.), Machine Learning and Knowledge Discovery in Databases, Springer Berlin Heidelberg, Berlin, Heidelberg, 2013, pp. 451–466.

[47] I. Tsamardinos, L.E. Brown, C.F. Aliferis, The max-min hill-climbing Bayesian network structure learning algorithm, Mach. Learn. 65 (2006) 31–78, http://dx.doi.org/10.1007/s10994-006-6889-7.

[48] J. Peters, P. Bühlmann, Structural intervention distance for evaluating causal graphs, Neural Comput. 27 (3) (2015) 771–799, http://dx.doi.org/10.1162/NECO_a_00708.

[49] K. Sachs, O. Perez, D. Pe'er, et al., Causal Protein-Signaling networks derived from Multiparameter Single-Cell data, Science 308 (5721) (2005) 523–529, http://dx.doi.org/10.1126/science.1105809.

[50] Y. Zheng, B. Huang, W. Chen, et al., Causal-learn: Causal discovery in python, J. Mach. Learn. Res. 25 (60) (2024) 1–8.

[51] T. Brooks, D. Pope, M. Marcolini, Airfoil Self-Noise, UCI Machine Learning Repository, 2014, http://dx.doi.org/10.24432/C5VW2C.

[52] F. Abid, Algerian Forest Fires, UCI Machine Learning Repository, 2019, http://dx.doi.org/10.24432/C5KW4N.

[53] J. Gerritsma, R. Onnink, A. Versluis, Yacht Hydrodynamics, UCI Machine Learning Repository, 2013, http://dx.doi.org/10.24432/C5XG7R.

[54] P. Cortez, A. Cerdeira, F. Almeida, et al., Modeling wine preferences by data mining from physicochemical properties, Decis. Support Syst. 47 (4) (2009) 547–553, http://dx.doi.org/10.1016/j.dss.2009.05.016, Smart Business Networks: Concepts and Empirical Evidence.

[55] R. Quinlan, Auto MPG, UCI Machine Learning Repository, 1993, http://dx.doi.org/10.24432/C5859H.

[56] D. Fuller, Replication Data for: Using machine learning methods to predict physical activity types with Apple Watch and Fitbit data using indirect calorimetry as the criterion, 2020, http://dx.doi.org/10.7910/DVN/ZS2Z2J.

[57] W. Nash, T. Sellers, S. Talbot, A. Cawthorn, W. Ford, Abalone, UCI Machine Learning Repository, 1995, http://dx.doi.org/10.24432/C55C7W.

[58] Iranian Churn, UCI Machine Learning Repository, 2020, http://dx.doi.org/10.24432/C5JW3Z.

[59] M.R. Hossain, Predicting customer churn in telecommunications with machine learning models, Asian J. Res. Comput. Sci. 18 (1) (2025) http://dx.doi.org/10.9734/ajrcos/2025/v18i1548.

[60] L. Castri, S. Mghames, M. Hanheide, N. Bellotto, Enhancing causal discovery from robot sensor data in dynamic scenarios, in: Proceedings of the Conference on Causal Learning and Reasoning (CLeaR), 2023.